

ECE1747H Assignment 1: CPU Parallelism with Pthreads and MPI A Range-Limited Electrostatic N-Particle Simulation

Zihao Gong
Student #:1005036916
UTORid:gongziha

1. Hardware Resource

The machine I'm using has AMD Ryzen 7800X3D as my processor. Here is the specifications:

Cores and Threads: 8 identical cores with 16 threads. Has a homogeneous core design.

Base and Boost Clock:

Base Clock: 4.2 GHz

Max Boost Clock: Up to 5.0 GHz, providing high single-threaded performance.

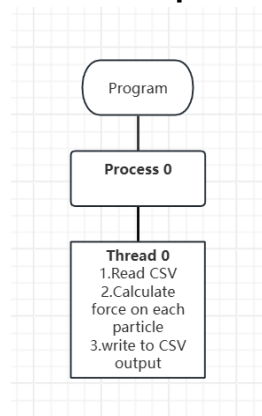
Cache:

L3 Cache: 96 MB (3D V-Cache), significantly enhancing gaming performance by minimizing latency.

TDP (Thermal Design Power):

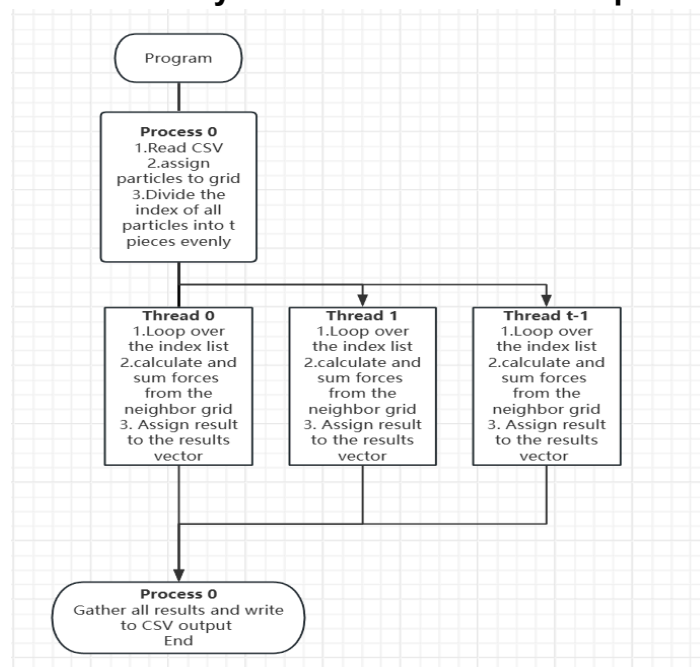
2. Program Architecture:

Mode 1:sequential computation



In my mode1, since we only have 1 process and no multi-thread here, so there's only one root thread that supports the process. Here the root thread will handle all the work. For all modes, I applied spatial partitioning that divides the data map into small grids by their location and stored in a hash map, the cutoff radius decides the grid size. So when we are calculating the force, we can directly access to the neighbor cells by the location key.

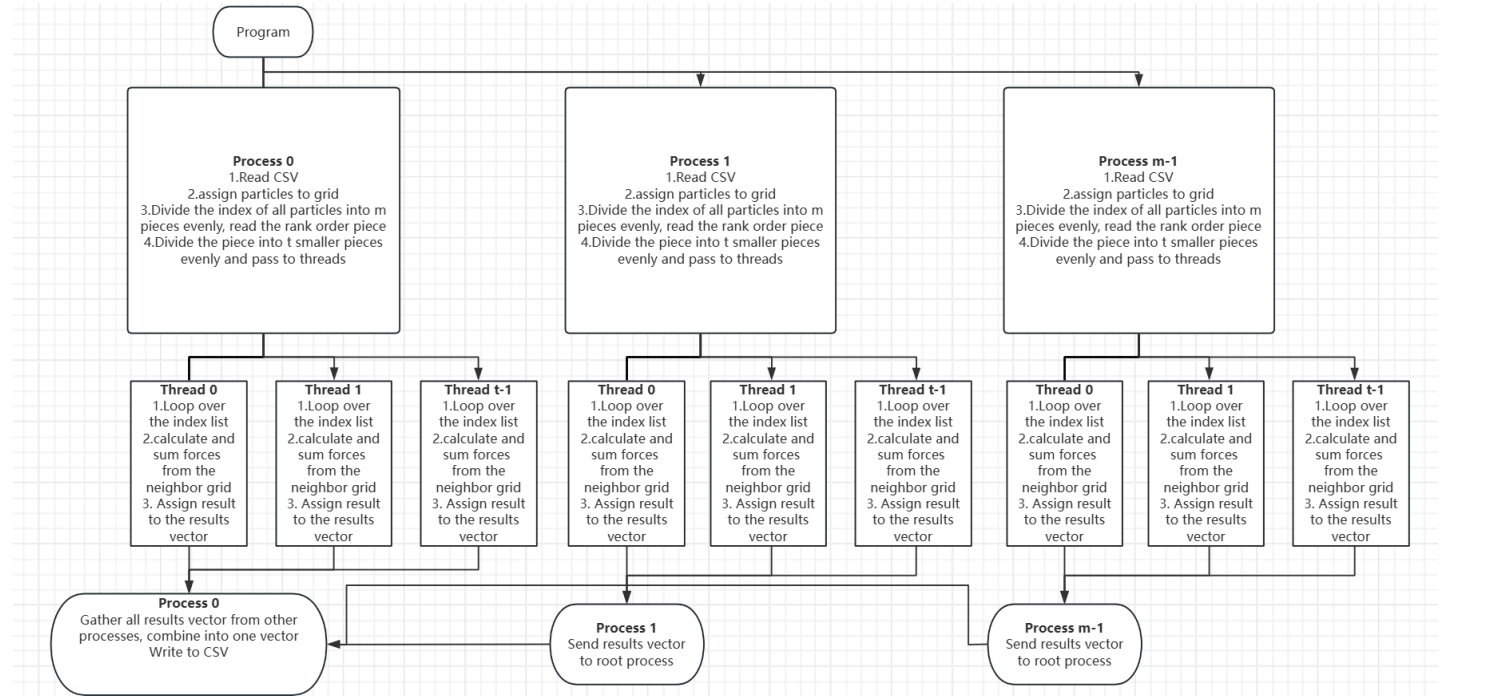
Mode 2:Evenly-Distributed Parallel Computation



In mode 2, We have a new parameter t, the number of threads. Therefore, in the block process 0, which is the root thread, will first do the serial job and before creating threads, divide all particles into pieces by

their index in the vector of dataset. If we have t threads, we'll divide the dataset in t pieces and first thread handle 0 to $t-1$, next solve t to $2t-1$ and so on. These data points are assigned like this to the threads. Each thread will calculate the whole force on the particles in the pieces just like mode 1. The summed force will be assigned to the results vector and after the parallel parts finish, the root thread will write to the CSV output file and calculate the error rate.

Mode3: Load-Balanced, Leader-Based Parallel Computation



In mode 3, we have another input parameter, the number of leader processes m . In each process, it'll read the CSV file and assign the grid separately. This work can be seen as serial because all processes need to do these steps first.

Then, it'll divide all particles into m parts and only take the rank order piece, then divide it further into smaller pieces (all by index to divide the work), and these smaller pieces of work become a queue,each threads will go pick the next piece after finishing the job till the queue is empty. After the calculation finished, each process got a results vector which is the results of the pieces with size total/m . Finally, the results are gathered and sent to the root process and reassembled into total size results. It will be written to a new CSV file. Here, all steps are serial again.

3. Mode 1: Sequential Computation:

After reading the CSV files and building the grid map, we'll loop over all the particles in the dataset to calculate the force on each particle. Here we'll just calculate the particles in the same or nearby grid to the target particle.

Cutoff radius (1e-10m)	1	100	1000	5000	10000	13000	18000
Avg error rate(%)	100	151.4	80.66	20.37	11.07	6.74	4.70

Then, we need to sweep the cutoff radius that achieves a <5% error. I tried a radius of 1,100,

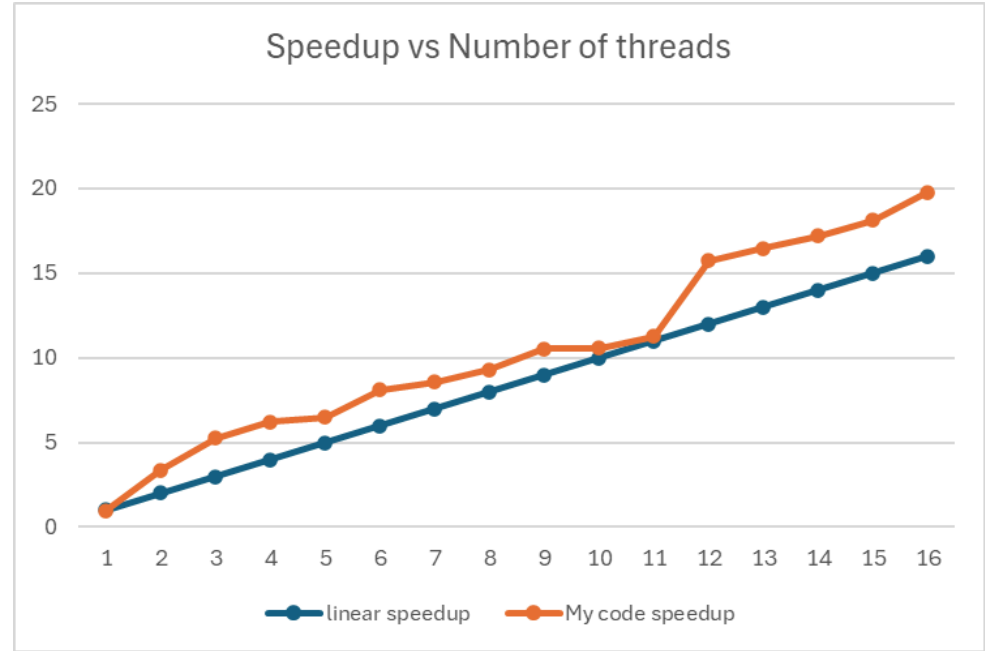
1000,5000,10000,13000,18000 and 21000.(The unit distance is 1e-10m.)

The error rate decreases rapidly from 1000 to 10000 and slows a lot after a 10000 radius. Finally, at 18000 radii I got an average error rate of 4.70%. The running time for radius **18000** is **1049.6 seconds**. However, the larger the cutoff radius we set, the lower the program runs. The average error rate versus the cutoff radius is like an exponential decrease plot.

4. Mode 2: Evenly-Distributed Parallel Computation:

1) I fixed the cutoff radius as 18000 to get <5% error rate.

Number of threads	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Time	1062.6	313.43	199.48	169.2	161.7	129.46	122.46	112.93	99.67	99.27	93.25	66.71	63.76	61.02	57.9	53.1
SpeedUp	0.98	3.35	5.26	6.20	6.49	8.10	8.57	9.29	10.53	10.57	11.26	15.73	16.46	17.2	18.13	19.77



From my result, the more threads we have, the more speed we can gain. The plot also shows it achieves superlinear in this case. The reason is we can leverage the core power with increasing # of threads and each thread will have smaller chunks of data to calculate. However, if the core power can't support so many threads it'll slow some threads' speed and in the end, other threads have to wait until the slow one is done, this can become a bottleneck that lowers our program. In my case, my core can support 16 threads and 16 threads always run better than 15 threads.

2).

The best combination I got is radius=17500 and 16 threads,

radius	17000	18000	18000	17700	17500	17300	17400	17450	17430
# of threads	16	16	15	16	16	16	16	16	16
time(s)	48.75	53	58.2	67.8	51.1	49.4	52.3	52.6	53.2
Error %	5.17	4.70	4.70	4.8	4.86	5.12	5.17	4.77	4.97

From my last tables and plot, I found **more threads** I have, **the faster** the speed is. For radius, if I **increase the radius** the **error rate will lower** but the trade-off is **speed lower**. So, since the question is finding the fastest execution time, my strategy is to take as many threads as I can and lower the radius till I get the 5% error boundary.

I focused on 3 radii that got good performance, 18000, 17500 and 17430 and I kept # of threads maximum. Then I repeated 5 times and got the average time for these focused radii.

18000:53.43,52.27,54.12,53.89,52.98 avg=53.34

17500:51.78,52.56,52.05,53.19,52.07 avg=52.33

17430:53.13,54.25,55.40,53.56,52.64 avg=53.8

That's the reason I chose the combo of **17500 radius and 16 threads**. The reason 17500 is better than 17430 maybe it just splits the data into the grid that more evenly and makes the calculation time more equal. The tradeoff is our error rate gets higher compared to the 18000 radius.

Mode3:

radius	17500	17500	17500	17500	18000	17430	17430	17450
# of threads	16	8	4	15	16	16	15	16
# of Leader process	1	2	4	1	1	1	1	1
time(s)	54.8.1 5	130.2 4	253.2	62.2	55.28	54.12	57.38	55.0

I'll keep my strategy from the last mode till I find the pattern. I keep the best radius of 17500 and only tune on the numbers of threads & leader processes. The multiplication of threads # and leader processes are always 16 since I want to leverage all of my device power.

Then I found that the less leader process we have, the faster the program runs. So I keep # of leader processes always 1, and try to tune the radius and the # of threads.

After trying some values, I found high thread # is still beneficial to runtime, while radius 17430 is better than the best result in mode 2. This makes sense cause a lower radius will have less data in one cell and lower the calculation amount. So, my final combo is **radius=17430,# of threads=16,# of leader process=1**.

6. Execution Time vs. Mode

Mode & parameters	1(radius=18000)	2(radius:18000,16 threads)	3(radius:18000,8 threads,2 processes)	3(radius:18000,16 threads,1 processes)
Read csv & parsing time(s)	0.458	0.46	0.487	0.435
Assign to grid time(s)	0.08	0.07	0.08	0.06
Data partition time(s)	/	0.44	0.45	0.37
Lowest thread Calculation time(s)	/	52.02	/	/
Total thread calculation time(s)	1047.7	57.3	157.2	58.68
Gather from processes time(s)	1049.6	58.99	159.6	59.28

My **experimental setup** is as follows:

For **all modes**, we tested on particles.csv and calculated the error rate with oracle.csv, using a hash table grid to store data nearby. The grid length & height are the same as the cutoff radius. The cutoff radius is always 18000. The radius will highly affect the mode performance. The lower the cutoff radius we set, the faster the program runs with a higher error rate.

For **mode 2**, I set 16 threads, the more threads I have, the higher speed-up I gain. This rule can be changed if we lower the radius. The reason is that compared to the cost of creating new threads, we can save more time with the huge amount of data (high cut-off radius) needed to be calculated.

For **mode 3** I set 2 processes and 8 threads per process, so the sum up is still 16 threads. I also set the number of smaller chunks to be $5 \times \text{number_threads}^2$. So that each thread can have a relatively equal workload. For this mode, if we can have a lower # of processes, we can get a higher speed. (With the same amount of threads in total). The reason is more processes we have, the more cost we'll spend on memory, communicating and creating processes.

Mode 2 is the fastest and Mode 1 is the slowest.

Mode 2 is the fastest since it achieves parallel for the most time-consuming parts and it doesn't have extra memory used for other processes to read the same data and no MPI communication cost. Even if I set Mode 3 that only have 1 process,

Mode 1 is slowest since it only uses 1 thread to calculate all data and no parallel speed-up can be done on Mode 1.

The difference between Mode 2 and Mode 3 with one process may be the additional overhead in managing the thread pool's tasks for Mode 3. The thread pool works well when the workload split is not equal and some threads are idle after finishing the job. However, for the **particles.csv** file and my CPU **homogeneous** core design, the time difference is only 2 seconds for threads to be idle. So, the speed will be different with different CPUs or datasets we're experimenting with.

7. Speedup

Superlinear refers to the situation where the speedup achieved by parallelizing the program exceeds the expected linear speedup based on the number of threads multiplied by the leader process number.

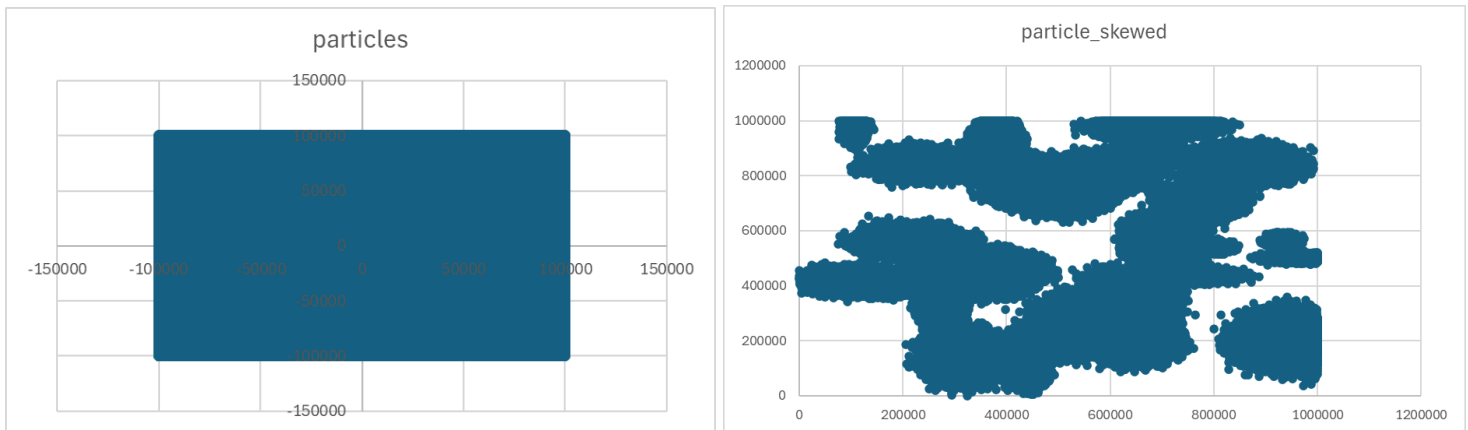
Superlinear is achievable due to these factors:

1. Serial coding parts in the program are negligible. This is the must condition for superlinear.
2. With more threads, the core may utilize the cache more effectively and reduce memory access time.
3. Dividing the work across cores might result in the data fitting into the faster memory hierarchy (like cache), resulting in superlinear speedup
4. If parallelization enables certain redundant computations to be eliminated, this can result in greater-than-expected speedups.

8. Re-usability

The hash table 'grid' was optimized to be calculated only once, its size is decided by radius, storing particle location data with the location as the key and the corresponding particle indices as values. This structure allows for efficient reusability, as it enables direct access to neighbour particles within a cutoff radius without recalculating the distances to all other particles. By reusing the precomputed grid data, the code avoids redundant computations, saving time and resources while ensuring that each particle can quickly retrieve relevant neighbour information for force calculations.

9. Bonus



For the skewed dataset, first, let's see the scatter diagram of the two datasets. The skewed dataset has the same amount of particles inside but with a larger distribution range. The normal dataset has a uniform dense everywhere while the skewed dataset has lots of sparse. So, I concluded the issues with the skewed dataset are:

1. For the particles at the edge, if we keep the old function that only looks for neighbours in 9 cells nearby we **can't find enough particles** and we'll have a **higher error rate** for these particles.
2. The calculation time varies for edge points and internal particles and this will cause Mode-2 threads' **workload not evenly distributed**. From my test, the fastest thread finished its work in 10 seconds while the slowest needed about 20 seconds to finish all the work. The 10s difference has lots of space for optimization.

So, I wrote a new function 'getMoreNeighbor' which will return the indices of far neighbour particles for edge and outlier particles. **This function identifies and returns a list of nearby particles within a specified range of cells, expanding outward from the central cell of the current particle. It does this by gradually checking cells within an expanding square radius until it reach the desired particle count or maximum distance.** With the desired neighbour particle count, we **lower the error for the edge particles** and **balance the workload** for edge points and internal particles.

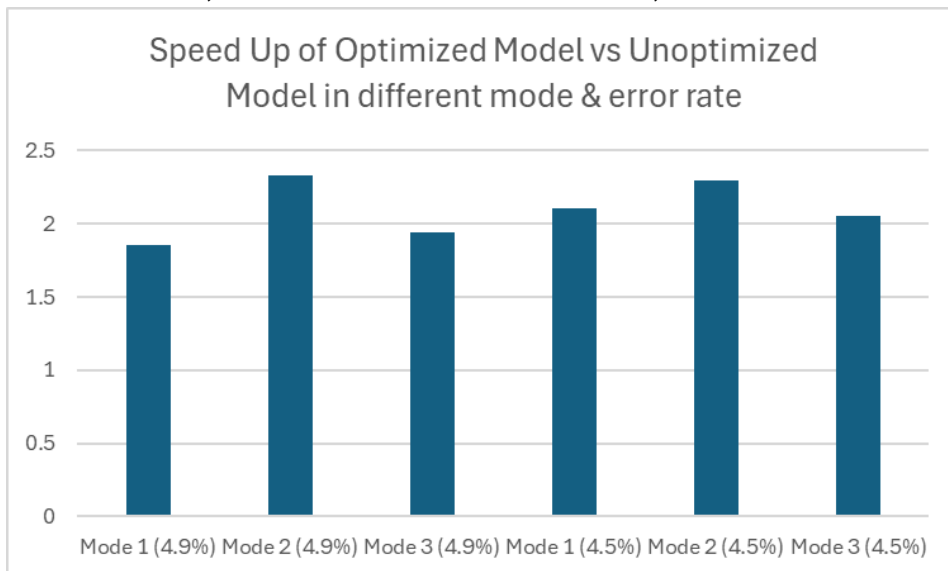
Since the function changes, there are lots of new independent parameters to change. Cell size, maximum distance, and desired number counts. For the cut-off radius, it just decided whether the neighbour point should participate in the force calculations and not influence the running speed like before. So, the cut-off radius should be higher and I pinned it to 45000. I tried to sweep max distance or cell size, but they don't affect a lot so I just pinned them to fix the value.

Optimized result:

Mode	1	2	3	1	2	3
Cell size	6000	6000	6000	6000	6000	6000
Max distance	5	5	5	5	5	5
Desired count	8000	8000	8000	6700	6700	6700
radius	45000	45000	45000	45000	45000	45000
time(s)	98.98	12	10.7	95.67	10.3	9.44
Error %	4.57	4.57	4.57	4.9	4.90	4.90

From experimental results, I found the time difference between the fastest and the slowest threads is still like 2 seconds. Then we can find that mode 3 works better since it further optimizes the workload evenly and therefore has a better result.

I also tested on my unoptimized code to get a close error rate. Pinned radius to 26000 so that we get a 4.85 error rate, the fastest mode 2 needs 24s, mode 3 needs 18.3s and mode 1 needs 177.48s.



This bar diagram shows that the optimized code has an average of 2 times speedup compared to the unoptimized code. The tradeoff is there are lots of independent parameters that can affect runtime and error rate and we need to manually try till we get our desired error rate.